CC-214 Computer Networks

Complete Solved Paper — Model Answer Key

QUESTION # 1 — Multiple Choice Questions (Part-I)

Each question carries 1 mark. The correct answer with explanation is provided below.

Q#	Question	Correct Answer	Explanation
1	UDP provides which reliability feature?	**C. Error detection**	UDP provides a basic checksum for error detection in its header. It does NOT provide retransmission, ordering, or flow control — those are TCP features. Note: The student chose D (Flow control) which is WRONG.
2	TCP is preferred for file transfer because it provides:	**B. Reliable, ordered delivery**	TCP guarantees that all bytes arrive in order without loss, which is essential for file integrity.
3	TCP flow control prevents:	**B. Receiver buffer overflow**	Flow control uses the receiver's advertised window (rwnd) to prevent the sender from overwhelming the receiver's buffer.
4	DNS primarily uses which transport protocol?	**A. UDP**	DNS uses UDP (port 53) for standard queries because it is fast and low-overhead. Responses are typically small enough to fit in one UDP datagram.
5	DNS uses TCP when:	**B. Response is large**	DNS switches to TCP when the response exceeds 512 bytes (traditional limit) or 4096 bytes (with EDNS). Zone transfers also use TCP. Note: The student chose A (Cache is used) which is WRONG.
6	DNS root servers:	**B. Know addresses of TLD servers**	Root servers do not store full mappings. They only know the addresses of Top-Level Domain (TLD) servers (e.g., .com, .org).
7	Which HTTP version uses QUIC?	**D. HTTP/3**	HTTP/3 is built on QUIC (a UDP-based transport protocol), replacing TCP to reduce latency and head-of-line blocking.
8	Which HTTP method retrieves a resource?	**A. GET**	GET is used to retrieve a resource from the server without modifying it. It is idempotent and safe.
9	Which HTTP status code indicates success?	**B. 200**	HTTP 200 OK is the standard success response. 301 = redirect, 404 = not found, 500 = server error.
10	Persistent HTTP improves performance by:	**B. Reducing connection setup overhead**	Persistent HTTP (HTTP/1.1 default) reuses a single TCP connection for multiple requests, avoiding repeated 3-way handshakes.
11	Open protocols ensure:	**C. Interoperability**	Open protocols (e.g., HTTP, SMTP) are publicly documented, allowing any vendor's systems to communicate with each other.
12	Which application is loss-tolerant?	**C. Video streaming**	Video streaming can tolerate some lost packets because occasional dropped frames are less noticeable than delay. File transfer, email, and banking require 100% reliability.

Q#	Question	Correct Answer	Explanation
13	Which application requires low latency but tolerates loss?	**B. Voice over IP (VoIP)**	VoIP needs low delay (< ~150ms) to sound natural but can tolerate a small percentage of packet loss.
14	Which application requires strict reliability?	**C. File transfer**	File transfer (FTP) requires every bit to arrive correctly — even one corrupted byte can corrupt the file.
15	NAT is usually implemented at:	**C. Network layer**	NAT operates at the Network layer (Layer 3), translating IP addresses. It also reads transport-layer port numbers (Layer 4) for NAPT.
16	Why does Selective Repeat outperform Go-Back-N in lossy networks?	**B. Fewer unnecessary retransmissions**	Go-Back-N retransmits ALL packets from the lost one onward. Selective Repeat retransmits ONLY the lost packet, saving bandwidth on lossy links.
17	Why must Selective Repeat window size be <= half of sequence space?	**B. Avoid ambiguity in packet identification**	If the window were larger than half the sequence space, the receiver could confuse a new packet with a retransmission of an old one.
18	Selective Repeat requires larger sequence number space because:	**B. Out-of-order packets must be distinguished**	SR buffers out-of-order packets, so each packet in flight (and its retransmission) must have a unique, distinguishable sequence number.
19	Which TCP mechanism prevents receiver buffer overflow?	**B. Flow control**	TCP flow control uses the receiver window (rwnd) field in the segment header so the sender never sends more than the receiver can buffer.
20	UDP demultiplexing differs from TCP because UDP uses:	**B. Destination port only**	UDP sockets are identified by (destination IP, destination port) only — any source can send to the same socket. TCP uses all 4 tuple values.
21	Which HTTP response indicates cached content is still valid?	**C. 304 Not Modified**	When a conditional GET finds the cached copy is still fresh, the server responds with 304 Not Modified (no body sent), saving bandwidth.
22	How does NAT differentiate flows with the same source port?	**C. By assigning unique external port numbers**	NAT (specifically NAPT) assigns a unique external port number to each internal connection, storing the mapping in the NAT translation table.
23	NAT causes problems with fragmented packets because:	**B. Each fragment may get different mappings**	Only the first fragment carries port numbers. Subsequent fragments lack port info, so NAT may map them differently or drop them.
24	NAT-based load balancing modifies:	**B. Destination IP address**	Load-balancing NAT rewrites the destination IP to distribute connections across multiple backend servers.
25	Pipelining improves protocol efficiency by:	**B. Allowing multiple unacknowledged packets**	Pipelining keeps the sender busy by transmitting multiple packets before waiting for ACKs, maximizing link utilization.
26	SMTP differs from HTTP mainly because SMTP is:	**C. Client-push**	SMTP is a push protocol — the sending mail server pushes the message to the receiving server. HTTP is primarily pull (client requests content).

Q#	Question	Correct Answer	Explanation
27	Iterative DNS queries differ from recursive queries because:	**A. Client contacts all servers directly**	In iterative resolution, each DNS server returns a referral; the client (or local resolver) contacts the next server itself. In recursive, the server does all the work.
28	Go-Back-N retransmits:	**B. All packets in the window**	Upon detecting a loss, Go-Back-N retransmits the lost packet AND all subsequent packets in the current window.
29	Congestion control in TCP reacts to:	**C. Network overload**	TCP congestion control (slow start, AIMD) responds to network-level congestion signals: timeout or duplicate ACKs.
30	UDP demultiplexing is based on:	**C. Destination port only**	UDP directs an incoming segment to the socket that matches the destination port number, regardless of source.
31	A conditional GET helps by:	**B. Avoiding unnecessary object transmission**	A conditional GET (using If-Modified-Since) lets the server send 304 Not Modified instead of re-sending an unchanged object.
32	HTTP/2 reduces head-of-line blocking by:	**C. Dividing objects into frames**	HTTP/2 breaks objects into binary frames and interleaves them over a single TCP connection, so a large object doesn't block smaller ones.
33	An HTTP response status code of 404 indicates:	**C. Object not found**	HTTP 404 Not Found means the requested resource does not exist on the server.
34	One performance concern of NAT is that it must recalculate:	**C. IP and TCP checksums**	Because NAT rewrites IP addresses (and ports), it must recompute the IP header checksum and TCP/UDP checksum on every translated packet.
35	The TCP timeout interval is calculated as:	**C. EstimatedRTT + 4 * DevRTT**	TCP sets TimeoutInterval = EstimatedRTT + 4 * DevRTT, providing a safety margin of 4 standard deviations above the mean RTT.
36	In a TCP segment header, the sequence number field represents:	**B. The byte-stream number of the first data byte in this segment**	TCP is byte-stream oriented. The sequence number identifies the position of the first data byte of this segment within the overall byte stream.
37	The Selective Repeat dilemma illustrates the need for the relationship:	**B. Window size <= (Sequence number space) / 2**	The SR window must be at most half the sequence number space to prevent old and new packets from being confused.
38	Which video encoding characteristic describes bit rate changing with scene complexity?	**B. VBR (Variable Bit Rate)**	VBR encoding adjusts the bit rate based on content complexity — high rate for action scenes, low rate for static scenes — improving quality-per-bit.
39	Which protocol is used by a user agent (e.g., Outlook) to retrieve e-mail from a server?	**B. IMAP**	IMAP (Internet Message Access Protocol) allows mail clients to retrieve and manage messages on the server. SMTP is for sending mail, not retrieving.

Q#	Question	Correct Answer	Explanation
40	Where is the cookie header line placed to set a user's ID on their browser?	**B. In the HTTP response message from the server**	The server uses the Set-Cookie header in its HTTP response to instruct the browser to store a cookie. The browser then sends it back via the Cookie header in future requests. Note: The student chose C (TCP handshake) which is WRONG.

QUESTION # 2 — Descriptive Questions (Part-II)

Time: 80 minutes | 5 × 8 = 40 marks

Part A — HTTP Statelessness and Cookies (8 marks)

Question: Susan browses an online bookstore built on standard HTTP. She adds a book on Page A (cart shows 1), browses to Page B, adds another book on Page B (cart still shows 1), then clicks View Cart on Page C (cart is empty). Explain why, and describe how cookies solve the problem.

Model Answer:

Why this happens — HTTP is stateless:

HTTP is a stateless protocol: each request–response pair is completely independent. The server retains no memory of previous interactions with the same client.

Step-by-step trace:

Step 1 — Page A, Add to Cart:
Susan's browser sends HTTP GET /pageA. The server responds and the browser shows the page. When she clicks 'Add to Cart', a new HTTP POST/GET request is sent. The server processes it and returns '1 item in cart' — but stores NO session state. The cart data is only in the response.

Step 2 — Navigate to Page B:
Browser sends HTTP GET /pageB. This is a brand new, independent request. The server has no memory of the previous Add-to-Cart request, so it cannot know a book was added.

Step 3 — Page B, Add to Cart:
Another independent HTTP request is sent to add the second book. Again, the server has no context about the first addition. It can only see this isolated request.

Step 4 — View Cart on Page C:
HTTP GET /cart is a fresh request. The server has no stored session for Susan — it sees an anonymous request and returns an empty cart.

Solution — Cookies mechanism:

Cookies allow the server to create a persistent, client-side state identifier across multiple stateless HTTP requests. The four key components are:

- **Cookie header in HTTP response:** When Susan first visits the site, the server creates a unique session ID (e.g., sid=abc123) and sends it in the response: Set-Cookie: sid=abc123.
- **Cookie stored in browser:** The browser saves this cookie locally, associated with the website's domain.
- **Cookie header in HTTP request:** On every subsequent request to the same site, the browser automatically includes: Cookie: sid=abc123.
- **Server-side cookie database:** The server maps sid=abc123 to Susan's shopping cart data (stored in a database). Now every request carries her identity, and the server can retrieve her cart correctly.

With cookies, when Susan adds a book on Page A, the server stores {sid=abc123 → [Book1]} in its database. When she later views the cart on Page C, the Cookie header identifies her, and the server correctly returns all items.

Part B — Web Caching Analysis (8 marks)

Question: A university has a 1.54 Mbps access link. Average request rate = 20 req/s, object size = 200 KB. Access link is saturated. Propose a web caching solution and calculate the required hit rate.

Model Answer:

1. Problem Analysis:

- Traffic demand = 20 req/s × 200 KB = 20 × 200 × 8 Kbps = 32,000 Kbps = 32 Mbps
- Access link capacity = 1.54 Mbps
- Link utilization without cache = 32 Mbps / 1.54 Mbps ≈ 20.8 → link is heavily saturated (>>100%)
- This causes queuing delays and packet loss, explaining the high observed delays.

2. Proposed Solution — Install a local Web Cache / Proxy Server:

Place a web cache (proxy server) inside the university network. When a client requests an object: (1) the cache is checked first; (2) if found and fresh (cache HIT), the object is served locally over the fast LAN — no access link traffic; (3) if not found (cache MISS), the request goes through the 1.54 Mbps access link to the Internet.

3. Required Hit Rate Calculation:

For the access link not to be saturated, the traffic flowing through it must be less than 1.54 Mbps.

- Traffic through access link = (1 – hit rate) × 32 Mbps
- Requirement: (1 – h) × 32 Mbps < 1.54 Mbps
- (1 – h) < 1.54 / 32 = 0.04813
- h > 1 – 0.04813 = 0.9519
- Required hit rate > 95.2% for the access link to be unsaturated.
- At a 60.5% hit rate (as calculated in the paper): traffic = 0.395 × 32 = 12.64 Mbps — still over capacity.
- Note: The paper's calculation of 13/20 = 0.605 appears to be a worked example or given hit rate scenario.

4. Performance Improvements with Web Caching:

- Reduced access link utilization — cached requests never use the bottleneck link.
- Lower response time — cache hits served at LAN speed (~100 Mbps or more) vs. internet delay.
- Reduced cost — less ISP bandwidth consumed, potentially lowering subscription costs.
- Reduced load on origin servers — fewer requests reach the actual web server.
- Conditional GETs — the cache can validate objects with the server using If-Modified-Since, keeping content fresh without full re-downloads.

Part C — DASH Adaptive Streaming (8 marks)

Question: Maria streams a movie using DASH on her smartphone. She starts on home Wi-Fi then moves to cellular (lower bandwidth). Describe how the DASH client adapts, what decisions it makes independently, and what it gets from the server.

Model Answer:

DASH — Dynamic Adaptive Streaming over HTTP:

DASH is a technique where video is encoded at multiple quality levels (bit rates) and divided into short segments (typically 2–10 seconds each). The client dynamically chooses which quality segment to request based on measured network conditions.

Information the client receives FROM the server:

- Manifest file (MPD — Media Presentation Description): sent once at the start of the session. It lists all available bit rates, URLs for each segment at each quality level, and segment durations.
- Video segments: the actual encoded video chunks, served via standard HTTP GET requests.

Decisions the client makes INDEPENDENTLY:

- Which quality level (bit rate) to request for the next segment — based on measured available bandwidth.
- When to request the next segment — the client controls the download timing.
- Which CDN server to use — the manifest may list multiple server URLs; the client can choose the closest or fastest.
- Buffer management — the client decides when to pause downloading to avoid wasting bandwidth vs. risk of buffer starvation.

Scenario trace — Wi-Fi to Cellular:

On home Wi-Fi (high bandwidth ~20 Mbps): Client measures high throughput → requests high-quality segments (e.g., 4K or 1080p at 8 Mbps). Buffer fills quickly.

Transition to cellular (~3 Mbps): Client detects download rate has dropped (segment takes longer to arrive than expected). Buffer starts draining.

Adaptation: Client's rate adaptation algorithm (e.g., BOLA or throughput-based) selects a lower bit rate segment (e.g., 480p at 1.5 Mbps) that matches the available bandwidth.

Steady state on cellular: Client continuously measures throughput and adjusts quality up or down to fill the playback buffer without interruption.

The key insight: the server is a simple HTTP file server — it makes NO adaptation decisions. All intelligence resides in the client, making DASH highly scalable.

Part D — NAT and NAPT (8 marks)

Question: Home router uses 192.168.1.0/24 internally; public IP 108.195.4.119. Alice (192.168.1.101) and Bob (192.168.1.102) simultaneously browse the same website. How does NAT keep their responses separate?

Model Answer:

The Technique: NAPT (Network Address and Port Translation)

With only one public IP, the router uses NAPT — it replaces not just the IP address but also the source port number. Each outgoing connection gets a unique external port, allowing multiple private devices to share a single public IP.

NAT Translation Table (example):

Private IP	Private Port	Public IP	Public Port	Destination
192.168.1.101 (Alice)	5000 (e.g.)	108.195.4.119	40001	Web server
192.168.1.102 (Bob)	5000 (e.g.)	108.195.4.119	40002	Web server

How it works step by step:

1. Alice sends a TCP SYN to the web server. Router rewrites the source: 192.168.1.101:5000 → 108.195.4.119:40001. Stores the mapping.

2. Bob sends a TCP SYN. Router rewrites: 192.168.1.102:5000 → 108.195.4.119:40002. Stores separately.

3. Web server sees two connections from the SAME IP (108.195.4.119) but DIFFERENT ports (40001 and 40002). It treats them as distinct connections.

4. Web server replies to 108.195.4.119:40001. Router looks up the table: destination port 40001 → forward to 192.168.1.101:5000 (Alice). Rewrites destination accordingly.

5. Web server replies to 108.195.4.119:40002. Router maps port 40002 → 192.168.1.102:5000 (Bob). Correct delivery.

The router must also recalculate IP and TCP checksums after modifying the address/port fields.

Part E — TCP Demultiplexing (8 marks)

Question: Apache server at 93.184.216.34 port 80. Client A (192.168.1.5, port 55000) and Client B (10.0.0.7, port 62000) both send TCP SYN packets. How does TCP demultiplex the segments to the correct process?

Model Answer:

TCP Demultiplexing using the 4-tuple:

Unlike UDP (which uses only destination port), TCP identifies each connection using a unique 4-tuple: (Source IP, Source Port, Destination IP, Destination Port). Even though both clients connect to the same destination (93.184.216.34:80), their source IPs and ports differ.

Connection identification:

Connection	Source IP	Source Port	Dest IP	Dest Port	4-tuple unique?
Client A	192.168.1.5	55000	93.184.216.34	80	YES
Client B	10.0.0.7	62000	93.184.216.34	80	YES

Mechanism — how Apache handles this:

Welcoming socket: Apache has a listening socket bound to port 80. When a TCP SYN arrives, the OS accepts it and creates a NEW dedicated socket for that specific connection.

Socket for Client A: Socket A is bound to the 4-tuple (192.168.1.5:55000, 93.184.216.34:80). All segments from Client A match this socket.

Socket for Client B: Socket B is bound to (10.0.0.7:62000, 93.184.216.34:80). Segments from Client B match only this socket.

Incoming segment processing: When a TCP segment arrives, the kernel's transport layer extracts all four fields. It looks up the socket table and delivers the segment to whichever socket matches the 4-tuple exactly.

Delivery to application: Apache may spawn a thread or process for each connection. The segment data is placed in that connection's receive buffer, and the application reads from it via the socket API.

Key insight: Port 80 on the server is the WELCOMING port for new connection requests only. Once a TCP connection is established, communication happens through a dedicated socket identified by the complete 4-tuple — NOT just the port number. This is fundamentally different from UDP.